Kalle: A Transparent Language Model Using Kernel Ridge Regression with Random Fourier Features

Mathias Leonhardt
KI-Mathias / pmagentur.com, Hamburg, Germany
mathias-leonhardt@gmx.de

https://ki-mathias.de

April 2026
Version 2.0 (adds §6 on scalable training via absorber-stochasticization)

Abstract

We present Kalle, a fully client-side bilingual chatbot that performs next-word prediction and retrieval-augmented generation (RAG) using Kernel Ridge Regression (KRR) with Random Fourier Features—without neural networks, backpropagation, or gradient descent. The system makes the mathematical foundations of modern language models *transparent*: the same eigenvalue structure that governs convergence of the Neumann series, stability of PageRank, light transport in radiosity, and learning dynamics of neural networks is exposed in an interactive, inspectable system. Kalle serves 2,178 curated Q&A pairs bilingually, runs entirely in the browser via WebGL GPU (TensorFlow.js), and achieves functional conversational ability including multi-turn context and RAG—all through a single closed-form matrix solve. We derive the complete mathematical chain from projection to kernel methods and show that the Neural Tangent Kernel theorem [Jacot et al., 2018] makes this connection rigorous: in the infinite-width limit, a neural network *becomes* KRR. Version 2 of this paper (§6) adds a scalable training path: we reformulate the KRR system as power iteration on an absorber-stochasticized matrix—the same construction Google uses for PageRank, with the ridge parameter λ playing the role of the damping factor d—and provide a Block Preconditioned Conjugate Gradient (Block-PCG) implementation that is mathematically equivalent to the direct solve but scales to regimes where $O(D^3)$ Gaussian elimination becomes infeasible.

Demo: https://pmmathias.github.io/krr-chat/
Code: https://github.com/pmmathias/krr-chat
Blog: https://ki-mathias.de/en/krr-chat-explained.html

1 Introduction

Modern large language models solve a deceptively simple task: given a sequence of words, predict the next one. The mechanism—transformer attention over billions of parameters, trained via backpropagation with RLHF—is powerful but opaque. A student can learn *that* GPT-4 works; understanding *why* requires infrastructure and expertise that most curricula cannot provide.

We ask: **Can next-word prediction be solved with mathematics transparent enough that every component maps to a textbook concept—while remaining functional enough to demonstrate the principles at work?**

We present Kalle, a system that answers affirmatively. The mathematical structure is not a simplification of neural language models—it is the *same* structure, made visible. The unifying thread is **eigenvalues**: they control convergence of iterative methods (§2), determine what a model learns versus ignores (§2.4), govern the stability of PageRank and radiosity (§3), and—through the Neural Tangent Kernel—connect KRR to neural networks rigorously (§4).

2 Mathematical Foundations

2.1 From Projection to the Normal Equation

The simplest prediction problem: given data matrix X and targets $\mathbf{y}$, find the coefficient vector $\mathbf{c}$ that minimizes $\|X\mathbf{c} - \mathbf{y}\|^2$. The solution is the orthogonal projection of $\mathbf{y}$ onto the column space of X:

$$\hat{\mathbf{y}} = X(X^\top X)^{-1}X^\top \mathbf{y} \tag{1}$$

This requires solving the **normal equation**:

$$X^\top X\,\mathbf{c} = X^\top \mathbf{y} \quad \Longrightarrow \quad \mathbf{c} = (X^\top X)^{-1}X^\top \mathbf{y} \tag{2}$$

2.2 Iterative Solution and the Neumann Series

Instead of inverting $X^\top X$ directly, we can iterate. Starting from $\hat{\mathbf{y}}_0 = \mathbf{0}$:

$$\hat{\mathbf{y}}_{n+1} = \hat{\mathbf{y}}_n + X^\top(\mathbf{y} - X\hat{\mathbf{y}}_n) \tag{3}$$

The residual after n steps satisfies:

$$\mathbf{r}_n = (I - X^\top X)^n \cdot \mathbf{y} \tag{4}$$

This converges when all eigenvalues of $(I - X^\top X)$ have magnitude less than 1. The limit is the **Neumann series**—the matrix generalization of the geometric series $\sum q^k = 1/(1-q)$:

$$\sum_{k=0}^{\infty} A^k = (I - A)^{-1} \quad \text{when } \rho(A) < 1 \tag{5}$$

where $\rho(A)$ denotes the spectral radius. The iteration converges exactly to the closed-form solution (2).

2.3 Eigenvalues Control Everything

Let $\mu_1, \ldots, \mu_n$ be the eigenvalues of $X^\top X$ with corresponding eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$. The spectral decomposition gives $A^n = V\Lambda^n V^{-1}$. The component of the solution along the i-th eigenvector is retained after n iterations by:

$$f_i^{\text{iter}}(n) = 1 - (1 - \mu_i)^n \tag{6}$$

This has profound consequences: large μ_i (strong signal) yields $f_i \to 1$ quickly, while small μ_i (noise) gives $f_i \approx n\mu_i$—learned slowly. Early stopping at step n suppresses components with $\mu_i \ll 1/n$. Early stopping is not a heuristic—it is a *spectral filter* on the eigenvalue decomposition.

2.4 Ridge Regression: The Closed-Form Spectral Filter

Instead of stopping early, we add an explicit penalty. Ridge regression solves:

$$\mathbf{c}_\lambda = (X^\top X + \lambda I)^{-1}X^\top \mathbf{y} \tag{7}$$

The filter factor for the i-th eigencomponent becomes:

$$f_i^{\text{ridge}}(\lambda) = \frac{\mu_i}{\mu_i + \lambda} \tag{8}$$

Theorem 1 (Early Stopping $\approx$ Ridge Regression)**.** *The iterative filter* (6) *and the ridge filter* (8) *produce equivalent spectral filtering with the correspondence $\lambda \approx 1/n$. Both separate signal (large μ_i) from noise (small μ_i) using the eigenvalue spectrum.*

Eigenvalue μ_i	Ridge $\frac{\mu_i}{\mu_i+\lambda}$	Iter. $1-(1-\mu_i)^n$	Interpretation
1.0	0.999	1.000	Signal: passes through
0.1	0.909	0.651	Moderate: partially filtered
0.01	0.500	0.096	Noise: heavily suppressed
0.001	0.091	0.010	Noise: nearly eliminated

Table 1: Spectral filter comparison ($\lambda = 0.01$, $n = 10$).

2.5 The Kernel Trick: From Finite to Infinite Dimensions

Real-world data is rarely linear. The kernel trick lifts data into a higher-dimensional feature space $\phi : \mathbb{R}^d \to \mathcal{H}$ where nonlinear patterns become linear, without ever computing ϕ explicitly. A **kernel function** computes the inner product directly:

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle \tag{9}$$

The **Gaussian (RBF) kernel**

$$k(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) \tag{10}$$

has an infinite-dimensional feature space, yet the kernel matrix $K \in \mathbb{R}^{n\times n}$ remains finite. The iterative update generalizes to $\hat{\mathbf{y}}_{n+1} = \hat{\mathbf{y}}_n + K(\mathbf{y} - \hat{\mathbf{y}}_n)$—replace $X^\top X$ with K, same algorithm, same eigenvalue structure, but with infinite expressive power.

Kernel Ridge Regression yields:

$$\boldsymbol{\alpha} = (K + \lambda I)^{-1}\mathbf{y} \tag{11}$$

Prediction for a new point uses the **Representer Theorem** [Kimeldorf & Wahba, 1970]:

$$f(x_*) = \sum_{i=1}^{n} \alpha_i \, k(x_i, x_*) \tag{12}$$

The training data *is* the model.

2.6 Random Fourier Features: Making Kernels Scalable

The kernel matrix K is $n \times n$. For $n = 322{,}636$ training samples, this is infeasible. Rahimi & Recht [2007] showed that shift-invariant kernels can be approximated via random projection, using **Bochner's theorem**: a bounded continuous shift-invariant kernel is the Fourier transform of a non-negative measure. For the Gaussian kernel:

$$z(x) = \sqrt{\frac{2}{D}} \cos(\omega^\top x + b) \tag{13}$$

where $\omega \sim \mathcal{N}(0, \sigma^{-2}I)$ and $b \sim \text{Uniform}(0, 2\pi)$. The key property: $z(x)^\top z(x') \approx k(x, x')$, improving with D. With RFF, the KRR solution becomes a standard linear system:

$$W = (Z^\top Z + \lambda I)^{-1} Z^\top Y \tag{14}$$

where $Z \in \mathbb{R}^{N\times D}$ is the RFF feature matrix. This is Eq. (7) again—the same equation, the same eigenvalue structure, at every level of the hierarchy.

3 The Unified Chain: One Equation, Four Applications

The equation $(I - A)^{-1}\mathbf{b} = \sum_{k=0}^{\infty} A^k \mathbf{b}$ appears in four seemingly unrelated domains. In each case, convergence is governed by the eigenvalues of A.

3.1 Radiosity: Light Bouncing Between Walls

The radiosity equation describes global illumination:

$$\mathbf{B} = \mathbf{E} + \rho F \mathbf{B} \quad \Longrightarrow \quad \mathbf{B} = (I - \rho F)^{-1}\mathbf{E} = \sum_{k=0}^{\infty} (\rho F)^k \mathbf{E} \tag{15}$$

where $\mathbf{E}$ is direct emission, ρ is reflectivity, and F is the form factor matrix. The matrix ρF is **substochastic** (column sums < 1) because surfaces absorb energy. To make it stochastic, we add an **absorber dimension**—a virtual surface capturing unabsorbed light. The resulting stochastic matrix describes a Markov chain whose stationary distribution is the dominant eigenvector with eigenvalue 1.

3.2 PageRank: Importance Spreading Through Links

Google's PageRank [Page et al., 1998] applies the same structure. The **Google matrix**:

$$G = d \cdot M + \frac{1-d}{n} \mathbf{1}\mathbf{1}^\top \tag{16}$$

where M is the column-stochastic link matrix and $d \approx 0.85$ is the damping factor (analogous to ρ in radiosity). The PageRank vector is the dominant eigenvector: $\mathbf{r}^{(k+1)} = G\,\mathbf{r}^{(k)} \to \mathbf{r}^*$ with $G\mathbf{r}^* = \mathbf{r}^*$. The Perron-Frobenius theorem guarantees uniqueness; the spectral gap $1 - |\lambda_2|$ determines convergence speed.

3.3 KRR Language Model

Our language model has the same structure. The regularized prediction $\boldsymbol{\alpha} = (K + \lambda I)^{-1}\mathbf{y}$ can be rewritten as a Neumann series:

$$\boldsymbol{\alpha} = \frac{1}{\lambda} \sum_{k=0}^{\infty} \left(-\frac{K}{\lambda}\right)^k \mathbf{y} \tag{17}$$

Each term represents one "bounce" of prediction influence through the kernel similarity graph.

Domain	Matrix A	"Source" $\mathbf{b}$	Eigenvalue role
Regression	$X^\top X$	$X^\top \mathbf{y}$	Convergence speed
Radiosity	ρF	$\mathbf{E}$	Light distribution
PageRank	G	uniform start	Webpage ranking
KRR	K/λ	$\mathbf{y}/\lambda$	What model learns

Table 2: Four applications of the Neumann series $(I - A)^{-1}\mathbf{b}$.

4 The Neural Network Connection

4.1 Universal Approximation

Three results establish the power of our approach: (1) the **Representer Theorem** [Kimeldorf & Wahba, 1970]: the optimal KRR solution has finitely many coefficients regardless of kernel space dimension; (2) **Universal Approximation** [Cybenko, 1989]: a neural network with one hidden layer approximates any continuous function; (3) **Gaussian Kernel Universality** [Micchelli et al., 2006]: KRR with the Gaussian kernel can do the same. Both KRR and neural networks are dense in $C(K)$.

4.2 Neural Tangent Kernel: From Metaphor to Theorem

Jacot et al. [2018] proved that in the infinite-width limit, a neural network **becomes** kernel ridge regression. The **Neural Tangent Kernel** is:

$$\Theta(x, x') = \left\langle \frac{\partial f(x;\theta)}{\partial \theta}, \frac{\partial f(x';\theta)}{\partial \theta} \right\rangle \tag{18}$$

In the limit, Θ converges to a deterministic kernel and gradient descent converges to the KRR solution. The eigenvalues of Θ determine what the network learns first (large eigenvalues) and ignores (small eigenvalues). This is not a metaphor—Kalle's KRR system and a neural language model are governed by the **same eigenvalue structure**.

5 System Implementation

5.1 Training Pipeline

Training proceeds in closed form: (1) parse corpus (2,178 pairs, 64,532 tokens); (2) train Word2Vec embeddings (gensim, 32-dim, CBOW, 20 epochs); (3) encode 24-word contexts as concatenated position-weighted embeddings $\phi(x) \in \mathbb{R}^{768}$; (4) apply RFF with $D = 6{,}144$, $\sigma = 1.5$; (5) solve $W = (Z^\top Z + \lambda I)^{-1} Z^\top Y$ with $\lambda = 10^{-6}$, yielding $W \in \mathbb{R}^{6144\times 2977}$ ($\sim$18.1M parameters). Total training time: $\sim$3 minutes on a single CPU core; solve time: $\sim$2 seconds.

5.2 Inference

Prediction requires one matrix-vector multiplication per word: $\hat{w} = \arg\max(z(c)^\top W)$, running in <1 ms on WebGL GPU via TensorFlow.js.

5.3 Answer Retrieval and RAG

Kalle retrieves answers using combined scoring:

$$s(q, p) = 0.65 \cdot s_{\text{kw}}(q, p) + 0.35 \cdot s_{\text{sem}}(q, p) \tag{19}$$

where s_{kw} is IDF-weighted keyword overlap and s_{sem} is cosine similarity of 32-dim BoW+IDF embeddings. For domain questions, a RAG pipeline injects blog context as `kontext {chunk} frage {query}` and matches against context-conditioned pairs.

5.4 Prediction Comparison

For each word, the KRR model predicts its top-3 candidates. Agreement with the corpus is rendered **green** (verbatim knowledge); disagreement is rendered **yellow**. This makes the boundary between memorization and generalization *visible*.

6 Scalable Training via Absorber-Stochasticization

This section is new in Version 2.

The direct solve of Eq. (14) via `numpy.linalg.solve` requires $O(D^2)$ memory for the Gram-like matrix $Z^\top Z$ and $O(D^3)$ compute for Gaussian elimination. At Kalle's current scale ($D = 6144$) this is ~ 288 MB and ~ 2 seconds—manageable. At $D = 50{,}000$ it becomes ~ 20 GB and hours; at $D = 100{,}000$ it is infeasible on consumer hardware. To unlock this scalability regime, we reformulate the solve as an iterative scheme requiring only matrix-vector products.

6.1 The Absorber Construction: From Substochastic to Stochastic

Richardson iteration on the KRR system (7) has the fixed-point form

$$\boldsymbol{\alpha}_{n+1} = M\boldsymbol{\alpha}_n + \omega\mathbf{y}, \qquad M = I - \omega(K + \lambda I). \tag{20}$$

Choosing $\omega = 1/(1+\lambda)$ (after normalizing K so that $\mu_{\max}(K) \leq 1$) gives M a *substochastic* spectrum: all eigenvalues of M lie in $[0, 1)$, but row sums are less than one. This is the *exact* algebraic analogue of the substochasticity of ρF in the radiosity equation $\mathbf{B} = \mathbf{E} + \rho F\mathbf{B}$: both describe a process that loses mass at each iteration.

Following the PageRank/radiosity construction, we recover stochasticity by adding an absorber: let $\mathbf{a} = \mathbf{1} - M\mathbf{1}$ be the row-wise mass deficit and $\mathbf{p} = \omega\mathbf{y}/\|\omega\mathbf{y}\|_1$ be the normalized target. Then

$$\tilde{P} = M + \mathbf{a}\mathbf{p}^\top \tag{21}$$

is row-stochastic: $\tilde{P}\mathbf{1} = M\mathbf{1} + \mathbf{a} = \mathbf{1}$. **The ridge parameter λ plays the role of the PageRank damping factor d**: both stabilize the iteration by enforcing a spectral gap and thereby guaranteeing a unique dominant eigenvector (Perron-Frobenius).

The derivation (worked out in the accompanying technical note) shows that a pure rank-one augmentation is insufficient to encode the multi-output KRR solution as a single dominant eigenvector, but the construction nevertheless delivers the *conceptual* infrastructure: the spectral gap $1 - |\lambda_2(\tilde{P})| \geq \lambda/(1+\lambda)$ directly determines the convergence rate of any Krylov-subspace method operating on the same system.

6.2 Block Preconditioned Conjugate Gradient

For the practical implementation we use **Block Preconditioned Conjugate Gradient (Block-PCG)**, which inherits the Krylov subspace structure of the absorber construction but replaces power iteration with the provably optimal choice within that subspace. The cost structure is:

- **Per iteration**: one matrix-matrix product $A \cdot P$ where $A = Z^\top Z + \lambda I \in \mathbb{R}^{D\times D}$ and $P \in \mathbb{R}^{D\times V}$ is the search direction matrix (V = vocabulary size).
- **Convergence**: $O(\sqrt{\kappa(A)}\log(1/\epsilon))$ iterations to reach tolerance ϵ.
- **Memory**: $O(D \cdot V)$ for the state (X, R, Z, P); no LU factor required.

With a diagonal (Jacobi) preconditioner $\mathcal{P} = \mathrm{diag}(A)$, Block-PCG converged in 14 iterations on the full Kalle corpus ($D = 6144$, $V = 2977$), producing a weight matrix W numerically equivalent to the direct solve ($\|W_{\mathrm{PCG}} - W_{\mathrm{direct}}\|_F / \|W_{\mathrm{direct}}\|_F < 10^{-6}$). The downstream Top-1 accuracy differed only by sampling noise (62.8% vs. 63.5%).

6.3 Benchmarks

We measure in two regimes. First, synthetic SPD problems with structure $A = Z^\top Z + \lambda I$, $\lambda = 10^{-6}$, $V = 500$, to observe the scaling of CG iteration count with D (Table 3). Second, the actual Kalle training matrices reconstructed from the real corpus ($D = 6144$, $V = 2977$) with four back-ends: NumPy direct, NumPy Block-PCG, PyTorch CPU Block-PCG, and PyTorch MPS (Apple Silicon GPU) Block-PCG (Table 4).

Honest assessment. Three observations deserve emphasis:

1. **The bigger win is the BLAS back-end, not the GPU.** On this hardware, swapping NumPy for PyTorch (both on CPU) reduces Block-PCG's wall-clock by 4×; adding MPS on top contributes only an additional $\sim 5\%$. At $D = 6144$, the problem is still too small for the M-class GPU to cover the host-to-device transfer latency. We expect MPS to dominate at $D \geq 20{,}000$, where direct solve becomes memory-infeasible anyway.

D	Direct (s)	Direct mem	PCG iters	PCG (s)	PCG rel. err
256	0.001	1.0 MB	10	0.008	2.7×10^{-7}
512	0.003	2.0 MB	13	0.021	3.3×10^{-7}
1024	0.011	3.9 MB	18	0.070	6.2×10^{-7}
2048	0.061	7.8 MB	30	0.342	1.6×10^{-6}
4096	0.367	15.6 MB	38	1.379	2.2×10^{-6}

Table 3: Synthetic benchmarks at varying RFF dimension D. CG iteration counts grow sublinearly with D, consistent with the $O(\sqrt{\kappa})$ theoretical bound.

Solver (back-end, precision)	Time (ms)	Iterations	Rel. err vs direct
Direct (NumPy / LAPACK, Float64)	2097	1	reference
Block-PCG (NumPy, Float64)	6731	14	4.3×10^{-7}
Block-PCG (PyTorch CPU, Float32)	1704	11	1.0×10^{-5}
Block-PCG (PyTorch MPS / Apple GPU, Float32)	**1622**	11	1.0×10^{-5}

Table 4: Solver-only benchmarks on the real Kalle training matrices ($D = 6144$, $V = 2977$). The largest speedup comes from switching the BLAS back-end from NumPy to PyTorch (3.95×), not from the GPU per se (additional 1.05×).

2. **A well-implemented Block-PCG (PyTorch CPU) is already faster than the direct solve.** This inverts the naive expectation that iterative solvers are only worthwhile at very large D. With modern matrix-product libraries, the $O(\sqrt{\kappa}) \cdot O(D^2 \cdot V)$ cost of CG beats the $O(D^3)$ cost of Gaussian elimination already at Kalle's current scale.
3. **Float32 is sufficient.** The PyTorch solvers run in Float32 and differ from the Float64 direct reference by $\sim 10^{-5}$ in relative Frobenius norm. Downstream Top-1 accuracy is unaffected (within sampling noise): the argmax operation over score vectors is robust to this level of precision loss.

For completeness, we note that the linear solve represents only 2–7% of Kalle's total training time ($\sim$ 100 s end-to-end on this hardware). The remainder is dominated by streaming $Z^\top Z$ accumulation ($\sim$ 45 s) and HTML packaging/base64 encoding ($\sim$ 40 s). Block-PCG's value at this scale is therefore not a training-time reduction but rather the opening of a GPU scaling path; at larger D where the solve dominates, the speedup becomes primary. Kalle v2 ships with all three NumPy solvers (`src/build_v2.py -solver {direct,cg,power}`); the PyTorch variants are in `src/benchmark_real_kalle.py` as reference implementations.

6.4 Corpus-scaling test: does the solver share grow with more data?

A natural follow-up question is whether the 2–7% solve share is an artifact of Kalle's small corpus. We tested this by extending the training data with the full ki-mathias.de blog content (197K additional tokens, 30 articles in DE and EN), yielding three scale regimes at fixed $D = 6144$:

Scenario	N	V	Accum	Solve	Solve %
kalle (baseline)	323 K	2,977	45 s	2.4 s	5.0
kalle + blog	1.3 M	19,411	190 s	10.2 s	4.9
kalle + blog ×2	2.3 M	19,411	325 s	9.8 s	2.7

Table 5: Training-time decomposition as N (samples) grows by 7×. Accumulation scales linearly with N; solve scales with V (which jumped once from 2,977 to 19,411 and then stayed fixed). The solver's relative share remains essentially constant at 3–5%.

Table 5 reveals a structural property of the pipeline: the solve cost depends on V and D but not on N, while accumulation is $O(N \cdot D^2)$. The solver share therefore stabilizes rather than growing with corpus size. The lever that would make the solver dominant is D: at $D = 50{,}000$ the direct solve's $O(D^3)$ cost and $O(D^2)$ memory become prohibitive, and iterative methods become essential. An additional observation from the scaling test: with more diverse data (V rising from 2,977 to 19,411), the effective condition number of $Z^\top Z + \lambda I$ *improves*, and Block-PCG needed **only 7–8 iterations** instead of 20 — consistent with the $O(\sqrt{\kappa})$ theory and, encouragingly, the opposite of the naive expectation that larger problems are harder to iteratively solve.

7 Evaluation

7.1 Data Quality vs. Quantity

Iteration	Pairs	Encoding	Top-1
V1 (original)	57	Hash (128 buckets)	99.8%
V2 (mass)	4,301	Word2Vec + heuristics	34.9%
V3 (curated)	2,178	Word2Vec (32-dim)	63.5%

Table 6: Blind corpus expansion destroys quality; curation restores it.

7.2 Multi-Turn RAG Evaluation

A 4-turn conversation testing topic transitions and language switching achieved 3/4 correct chunk retrievals with kwRaw scores of 36.9–44.3 (high confidence).

7.3 Regression Test Suite

34 Playwright scenarios across greeting, food, emotion, math, meta, multi-turn, and edge cases, run automatically on every build.

8 Related Work

Kernel methods for NLP. String kernels [Lodhi et al., 2002] and spectrum kernels [Leslie et al., 2002] have been used for text classification. Our work applies KRR to autoregressive next-word prediction.

Random Fourier Features. Rahimi & Recht [2007]—NeurIPS Test of Time Award 2017—introduced RFF for scalable kernel approximation via Bochner's theorem.

Neural Tangent Kernel. Jacot et al. [2018] proved that infinitely wide neural networks converge to kernel regression.

Browser-based ML. Ma et al. [2019] evaluated TensorFlow.js for in-browser deep learning.

RAG. Lewis et al. [2020] introduced retrieval-augmented generation with neural components. Kalle demonstrates the RAG pattern is separable from neural architecture.

Iterative KRR solvers (Version 2). Richardson iteration for regularized regression goes back to Hestenes & Stiefel [1952] (Conjugate Gradient) and is analyzed as implicit regularization by Yao et al. [2007]. Scalable GPU-accelerated kernel machines via preconditioned CG were developed in Falkon [Rudi et al., 2017] and ASkotch [Rathore et al., 2024]. Nyström-based preconditioning [Frangella et al., 2023] and random-features preconditioning [Avron et al., 2017] are the current state of the art. Our contribution is the explicit translation of the PageRank damping construction [Page et al., 1998, Boldi et al., 2005] to KRR: interpreting λ as the damping factor of an absorber-stochasticized iteration matrix. The heat-kernel/PageRank duality [Chung,

2007] is conceptually adjacent but does not perform this stochasticization on the KRR normal equations.

9 Limitations

No free-form generation. Kalle retrieves pre-written answers. **Vocabulary-bound.** The 2,977-word vocabulary excludes most proper nouns. **Scaling ceiling.** The direct KRR solve requires $O(D^2)$ memory and $O(D^3)$ compute; the Block-PCG implementation in Version 2 (§6) relaxes both constraints but still requires $O(D \cdot V)$ state. **Preconditioner quality.** Our Block-PCG uses only a diagonal (Jacobi) preconditioner. Nyström or random-features preconditioning would likely yield a further 5–10× speedup at larger D; we leave this to future work. **Evaluation scope.** Formal metrics (perplexity, BLEU) are deferred to future work.

10 Conclusion

Kalle demonstrates that the mathematical structure underlying language models—eigenvalues, kernel functions, the Neumann series, regularization as spectral filtering—can be made transparent, interactive, and pedagogically useful. The same equation $(I - A)^{-1}\mathbf{b}$ governs light transport (radiosity), web ranking (PageRank), and prediction (KRR). The Neural Tangent Kernel theorem establishes that this is not a simplification of neural language models—it is the same mathematics, made visible.

Version 2 extends this unification from the *analytic* level to the *algorithmic* level. The absorber-stochasticization construction—hitherto used only to motivate the PageRank damping factor and to prove convergence of the radiosity iteration—reveals its full practical value when applied to Kernel Ridge Regression: the ridge parameter λ *is* the damping factor d, in a strong enough sense to substitute Gaussian elimination with Block-PCG and thereby open a path to GPU-scale training. The eigenvalue structure does not just explain *what* these systems compute—it also dictates *how* to compute them efficiently.

References

Cybenko, G. (1989). Approximation by Superpositions of a Sigmoidal Function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.

Jacot, A., Gabriel, F., & Hongler, C. (2018). Neural Tangent Kernel: Convergence and Generalization in Neural Networks. *NeurIPS*.

Kimeldorf, G. & Wahba, G. (1970). A Correspondence Between Bayesian Estimation on Stochastic Processes and Smoothing by Splines. *Annals of Mathematical Statistics*, 41(2):495–502.

Köpf, A., Kilcher, Y., von Rütte, D., et al. (2023). OpenAssistant Conversations—Democratizing Large Language Model Alignment. *NeurIPS Datasets and Benchmarks*.

Leslie, C., Eskin, E., & Noble, W. S. (2002). The Spectrum Kernel: A String Kernel for SVM Protein Classification. *PSB*.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.

Lodhi, H., Saunders, C., Shawe-Taylor, J., Cristianini, N., & Watkins, C. (2002). Text Classification using String Kernels. *JMLR*, 3:419–444.

Ma, Y., Xiang, D., Zheng, S., Tian, D., & Liu, Z. (2019). Moving Deep Learning into Web Browser: How Far Can We Go? *WWW*.

Micchelli, C. A., Xu, Y., & Zhang, H. (2006). Universal Kernels. *JMLR*, 7:2651–2667.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *ICLR Workshop*.

Page, L., Brin, S., Motwani, R., & Winograd, T. (1998). The PageRank Citation Ranking: Bringing Order to the Web. *Stanford Technical Report*.

Rahimi, A. & Recht, B. (2007). Random Features for Large-Scale Kernel Machines. *NeurIPS*.

Wallace, R. (2009). The Anatomy of A.L.I.C.E. In *Parsing the Turing Test*, Springer.

Wei, J., et al. (2022). Finetuned Language Models Are Zero-Shot Learners. *ICLR*.

Hestenes, M. & Stiefel, E. (1952). Methods of Conjugate Gradients for Solving Linear Systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436.

Yao, Y., Rosasco, L. & Caponnetto, A. (2007). On Early Stopping in Gradient Descent Learning. *Constructive Approximation*, 26(2):289–315.

Rudi, A., Carratino, L. & Rosasco, L. (2017). FALKON: An Optimal Large Scale Kernel Method. *NeurIPS*.

Rathore, A., Díaz, B., Frangella, Z. & Udell, M. (2024). Have ASkotch: A Neat Solution for Large-scale Kernel Ridge Regression. *arXiv:2407.10070*.

Frangella, Z., Tropp, J.A. & Udell, M. (2023). Randomized Nyström Preconditioning. *SIAM Journal on Matrix Analysis and Applications*.

Avron, H., Clarkson, K. & Woodruff, D. (2017). Faster Kernel Ridge Regression Using Sketching and Preconditioning. *SIAM Journal on Matrix Analysis and Applications*.

Boldi, P., Santini, M. & Vigna, S. (2005). PageRank as a Function of the Damping Factor. *WWW*.

Chung, F. (2007). The Heat Kernel as the PageRank of a Graph. *PNAS*, 104(50):19735–19740.